

Variables in C

- A **variable in C language** is the name associated with some memory location to store data of different types.
- There are many types of variables in C depending on the scope, storage class, lifetime, type of data they store, etc.
- A **variable** is the basic building block of a C program that can be used in expressions as a substitute in place of the value it stores.
- A **variable in C** is a memory location with some name that helps store some form of data and retrieves it when required.
- We can store different types of data in the variable and reuse the same variable for storing some other data any number of times.

C Variable Syntax

The syntax to declare a variable in C specifies the name and the type of the variable.

```
data_type variable_name = value; // defining single variable  
or  
data_type variable_name1, variable_name2; // defining multiple variable
```

Here,

- **data_type**: Type of data that a variable can store.
- **variable_name**: Name of the variable given by the user.
- **value**: value assigned to the variable by the user.

The diagram illustrates the syntax `data_type variable_name = value`. A horizontal bracket above the first part, `data_type variable_name`, is labeled *Definition* with an upward-pointing arrow. Another horizontal bracket above the second part, `= value`, is labeled *Initialization* with an upward-pointing arrow.

There are 3 aspects of defining a variable:

1. Variable Declaration
2. Variable Definition
3. Variable Initialization

→ C Variable Declaration

- Variable declaration in C tells the compiler about the existence of the variable with the given name and data type.
- No memory is allocated to a variable in the declaration.

→ C Variable Definition

- In the definition of a C variable, the compiler allocates some memory and some value to it.
- A defined variable will contain some random garbage value till it is not initialized.

Example:

```
int var;  
char var2;
```

→ C Variable Initialization

- Initialization of a variable is the process where the user assigns some meaningful value to the variable.

Example:

```
int var; // variable definition  
var = 10; // initialization  
    or  
int var = 10; // variable declaration and  
definition
```

Rules for Naming Variables in C

- A variable name must only contain alphabets, digits, and underscore.
- A variable name must start with an alphabet or an underscore only. It cannot start with a digit.
- No whitespace is allowed within the variable name.
- A variable name must not be any reserved word or keyword.

Types of Variables

The C variables can be classified into the following types:

- Local Variables
- Global Variables
- Static Variables
- Automatic Variables
- Extern Variables
- Register Variables

➔ Local Variables in C

- A **Local variable in C** is a variable that is declared inside a function or a block of code.
- Its scope is limited to the block or function in which it is declared.

➔ Global Variables in C

- A **Global variable in C** is a variable that is declared outside the function or a block of code.
- Its scope is the whole program i.e. we can access the global variable anywhere in the C program after it is declared.

➔ Static Variables in C

- A **static variable in C** is a variable that is defined using the **static** keyword. It can be defined only once in a C program and its scope depends upon the region where it is declared (can be **global or local**).
- The **default value** of static variables is **zero**.

➔ Automatic Variable in C

- All the **local** variables are **automatic** variables **by default**. They are also known as auto variables.
- Their scope is **local** and their lifetime is till the end of the **block**. If we need, we can use the **auto** keyword to define the auto variables.
- The default value of the auto variables is a garbage value.

➔ External Variables in C

- External variables in C can be shared between multiple C files. We can declare an external variable using the **extern** keyword.
- Their scope is global and they exist between multiple C files.

➔ Register Variables in C

- Register variables in C are those variables that are stored in the CPU register instead of the conventional storage place like RAM.
- Their scope is local and exists till the end of the block or a function.
- These variables are declared using the **register** keyword.
- The default value of register variables is a garbage value.

→ Example:

```
// C Program to Demonstrate Various types of variable

#include <stdio.h>

// Global variable
int globalVar = 10;

int main() {
    // Automatic (default) variable
    int autoVar = 5;

    // Local variable
    int localVar = 20;

    // Static variable
    static int staticVar = 15;

    // Extern variable (declared in another file)
    extern int externVar;

    // Register variable (not guaranteed to be stored in memory)
    register int regVar = 25;

    printf("Global variable: %d\n", globalVar);
    printf("Automatic variable: %d\n", autoVar);
    printf("Local variable: %d\n", localVar);
    printf("Static variable: %d\n", staticVar);
    printf("Extern variable: %d\n", externVar);
    printf("Register variable: %d\n", regVar);

    return 0;
}
```

Output:

```
Global variable: 10
Automatic variable: 5
Local variable: 20
Static variable: 15
```

➔ Constant Variable in C

- Till now we have only seen the variables whose values can be modified any number of times.
- C language also provides us a way to make the value of a variable immutable. We can do that by defining the variable as constant.
- A **constant variable** in C is a read-only variable whose value cannot be modified once it is defined. We can declare a constant variable using the **const** keyword.

Example:

```
// C Program to Demonstrate constant variable
#include <stdio.h>

int main()
{
    int not_constant;

    const int constant = 20;

    // changing values
    not_constant = 40;
    constant = 22;

    return 0;
}
```

Output:

```
Constant = 22
```